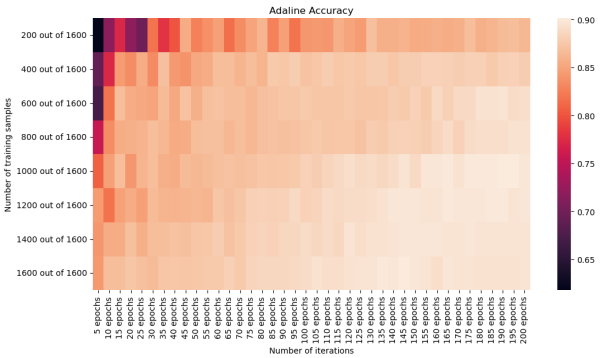
CA2 - Supervised machine learning classification pipeline - applied to medical data

Important information

- Do **not** use scikit-learn (`sklearn`) or any other high-level machine learning library for this CA
- Explain your code and reasoning in markdown cells or code comments
- Label all graphs and charts if applicable
- If you use code from the internet, make sure to reference it and explain it in your own words
- If you use additional function arguments, make sure to explain them in your own words
- Use the classes `Perceptron` , `Adaline` and `Logistic Regression` from the library `mlxtend` as classifiers (`from mlxtend.classifier import Perceptron, Adaline, LogisticRegression`). *Always* use the argument `minibatches=1` when instantiating an `Adaline` or `LogisticRegression` object. This makes the model use the gradient descent algorithm for training. Always use the `random_seed=42` argument when instantiating the classifiers. This will make your results reproducible.
- You can use any plotting library you want (e.g. `matplotlib` , `seaborn` , `plotly` , etc.)
- Use explanatory variable names (e.g. `X_train` and `X_train_scaled` for the training data before and after scaling, respectively)
- The dataset is provided in the file `fetal_health.csv` in the `assets` folder

Additional clues

- Use the `pandas` library for initial data inspection and preprocessing
- Before training the classifiers, convert the data to raw `numpy` arrays
- For Part IV, you are aiming to create a plot that looks similar to this:



Additional information

- Feel free to create additional code or markdown cells if you think it will help you explain your reasoning or structure your code (you don't have to).

Part I: Data loading and data exploration

Import necessary libraries/modules:

In [25]:
```
# Insert your code below
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
```

Loading and exploring data

1. Load the dataset `fetal_health.csv` with `pandas`. Use the first column as the row index.
2. Check for missing data, report on your finding and remove samples with missing data, if you find any.
3. Display the raw data with appropriate plots/outputs and inspect it. Describe the distributions of the values of feature `"baseline value"`, `"accelerations"`, and the target variable `"fetal_health"`.
4. Will it be beneficial to scale the data? Why or why not?
5. Is the data linearly separable using a combination of any two pairs of features? Can we expect an accuracy close to 100% from a linear classifier?

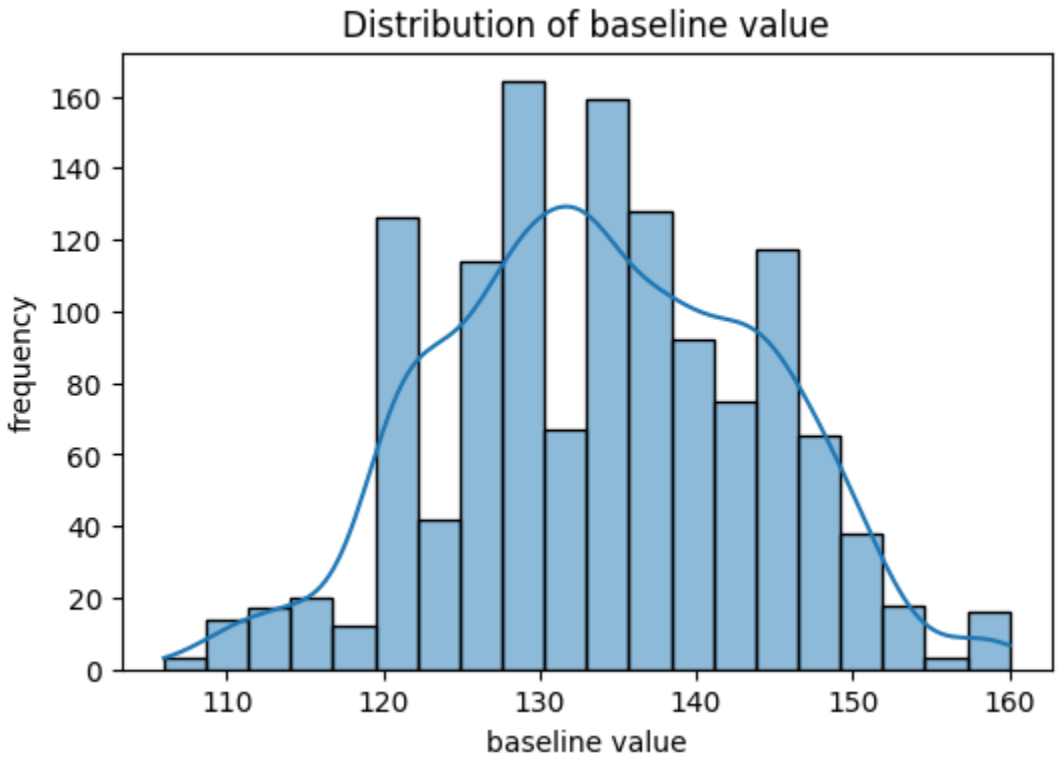
In [26]:
```
# Insert your code below
#part1
loadDF = pd.read_csv('assets/fetal_health.csv', index_col=0)
#part2
missing_data = loadDF.isnull().sum()
if missing_data.any():
    print("Missing data found in the dataset")
else:
    print("No missing data found in the dataset")
#remove the samples with missing data
loadDF_remove_missing = loadDF.dropna()
missing_data_after_remove = loadDF_remove_missing.isnull().sum()
print("Missing data in column after processing: \n", missing_data_after_remo
#part3
#for baseline value dist
plt.figure(figsize=(6,4))
sns.histplot(loadDF_remove_missing['baseline value'], kde=True)
plt.title('Distribution of baseline value')
plt.xlabel('baseline value')
```

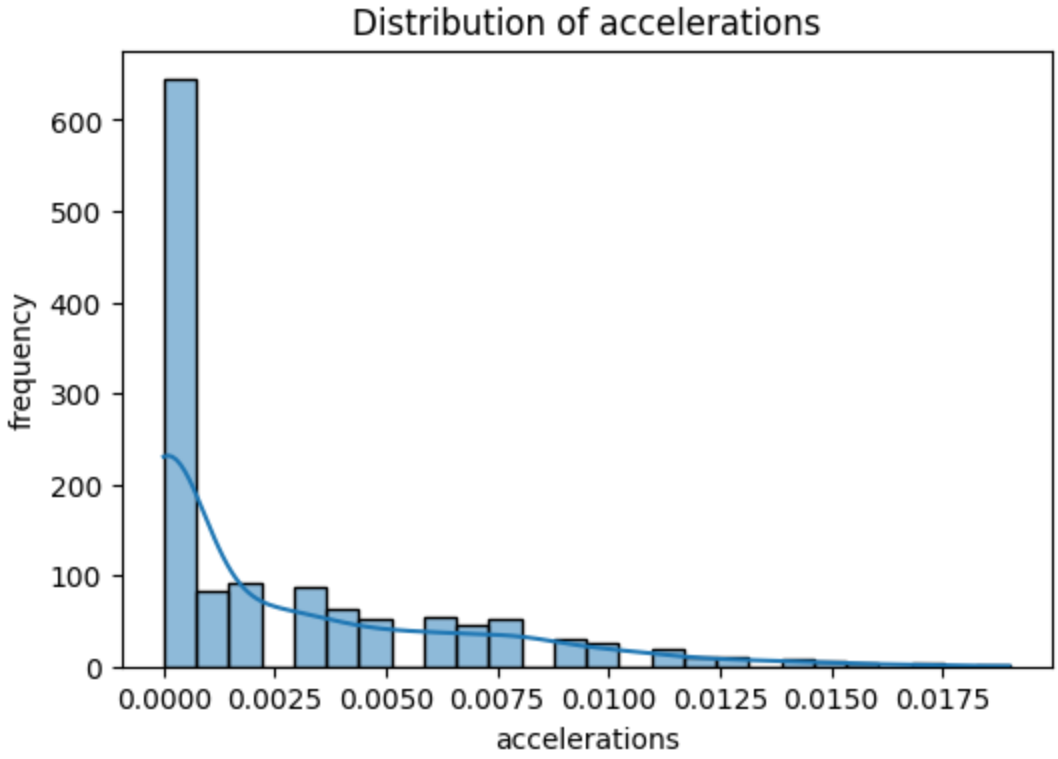
```
plt.ylabel('frequency')
plt.show()

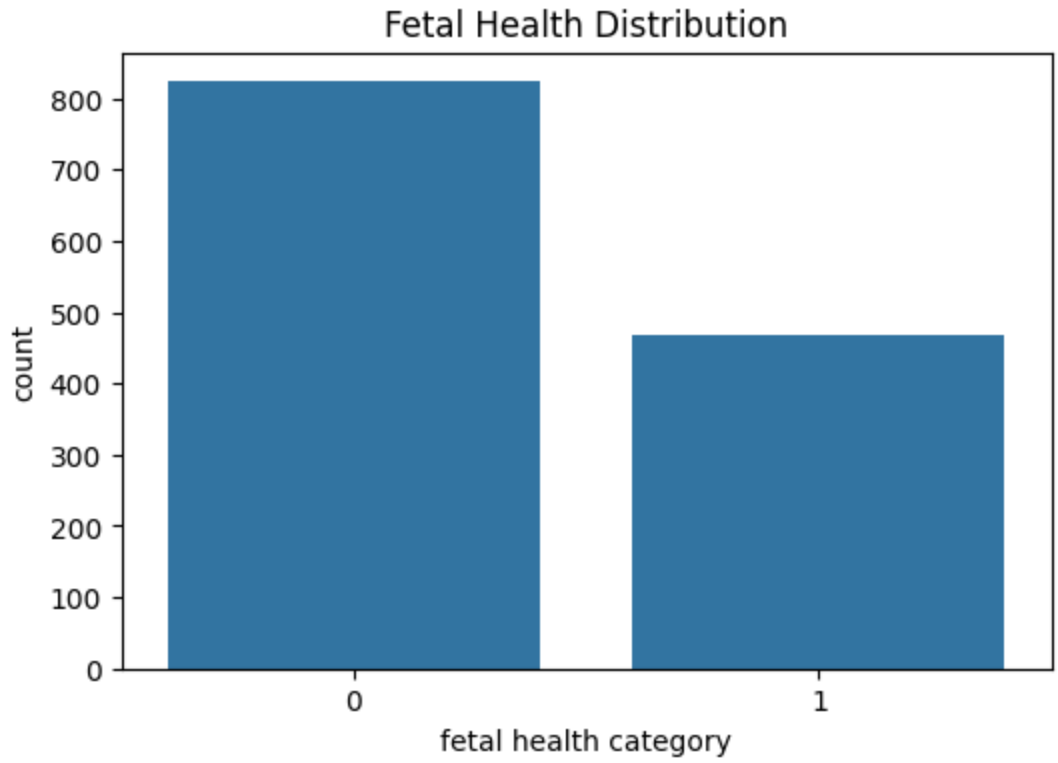
#for acceleration dist
plt.figure(figsize=(6,4))
sns.histplot(loadDF_remove_missing['accelerations'], kde=True)
plt.title('Distribution of accelerations')
plt.xlabel('accelerations')
plt.ylabel('frequency')
plt.show()

#for the target variable (fetal health distribution)
plt.figure(figsize=(6,4))
sns.countplot(x='fetal_health', data=loadDF_remove_missing)
plt.title('Fetal Health Distribution')
plt.xlabel('fetal health category')
plt.ylabel('count')
plt.show()
```

```
No missing data found in the dataset
Missing data in column after processing:
 baseline value                                            0
accelerations                                              0
prolongued_decelerations                                   0
abnormal_short_term_variability                            0
mean_value_of_short_term_variability                       0
percentage_of_time_with_abnormal_long_term_variability     0
histogram_mean                                             0
histogram_variance                                         0
fetal_health                                               0
dtype: int64
```







3. A. Distribution of baseline value:
    - The baseline value has a normal distribution
    - The values spread fairly symmetrically around the center at ca.130
    - there's a peak around 130-135

   B. Distribution of accelerations:
    - The distribution of accelerations is highly skewed to the right , with most

of the values at around 0
- the graph declines towards higher values
- Is not as informative because of the high concentration around 0.

C. Distribution of fetal health, the target variable
- Categories are imbalanced with a higher number healthy than unhealthy.
- Healthy has around 800 samples while unhealthy has about half.
- because of imbalance, models may favour predicting the majority class(healthy(0)), over the minor(unhealthy(1)) class.

4.

- because the baseline value distribution is a fairly normally distribution it does not require any transformation, altough scaling could prove benefical for ensuring all features cotribute the same to the model
- Scaling could prove useful in making the distribution of accelerations more symmetrical. Log transformation to be specific imo.
- class weighing or resampling may be necessary to ensure that both classes perform good in the distribution of fetal health, the target variable.

5.

- Based on the histograms we have gotten, the data seems to have overlapping distributions for the different features suggesting that a linear classifier will most probably not achieve near perfect accuracy. Due to overlapping between classes if used as a feature(baseline value dist), skewness(accelerations distribution), and non linear patterns.

Part II: Train/Test Split

Divide your dataset into training and testing subsets. Follow these steps to create the split:

1. **Divide the dataset into two data sets, each data set only contains samples of either class 0 or class 1:**

- Create a DataFrame `df_0` containing all data with `"fetal_health"` equal to 0.
- Create a DataFrame `df_1` containing all data with `"fetal_health"` equal to 1.

2. **Split into training and test set by randomly sampling entries from the data frames:**

- Create a DataFrame `df_0_train` containing by sampling `75%` of the entries from `df_0` (use the `sample` method of the data frame, fix the `random_state`

to 42).
- Create a DataFrame `df_1_train` using the same approach with `df_1` .
- Create a DataFrame `df_0_test` containing the remaining entries of `df_0` (use `df_0.drop(df_0_train.index)` to drop all entries except the previously extracted ones).
- Create a DataFrame `df_1_test` using the same approach with `df_1` .

3. **Merge the datasets split by classes back together:**

- Create a DataFrame `df_train` containing all entries from `df_0_train` and `df_1_train` . (Hint: use the `concat` method you know from CA1)
- Create a DataFrame `df_test` containing all entries from the two test sets.

4. **Create the following data frames from these splits:**

- `X_train` : Contains all columns of `df_train` except for the target feature `"fetal_health"`
- `X_test` : Contains all columns of `df_test` except for the target feature `"fetal_health"`
- `y_train` : Contains only the target feature `"fetal_health"` for all samples in the training set
- `y_test` : Contains only the target feature `"fetal_health"` for all samples in the test set

5. **Check that your sets have the expected sizes/shape by printing number of rows and colums ("shape") of the data sets.**

- (Sanity check: there should be 8 features, almost 1000 samples in the training set and slightly more than 300 samples in the test set.)

6. **Explain the purpose of this slightly complicated procedure. Why did we first split into the two classes? Why did we then split into a training and a testing set?**

7. **What is the share (in percent) of samples with class 0 label in test and training set, and in the intial data set?**

In [14]:
```
# Insert your code below
# ======================

# 1.
df_0 = loadDF[loadDF['fetal_health'] == 0]
df_1 = loadDF[loadDF['fetal_health'] == 1]

# 2. Training and Testing sets
df_0_train = df_0.sample(frac=0.75, random_state=42)
df_1_train = df_1.sample(frac=0.75, random_state=42)
```

```
df_0_test = df_0.drop(df_0_train.index)
df_1_test = df_1.drop(df_1_train.index)

# 3. Merge back together
df_train = pd.concat([df_0_train, df_1_train])
df_test = pd.concat([df_0_test, df_1_test])

# 4.
X_train = df_train.drop('fetal_health', axis=1)  # axis=1 to drop the column
X_test = df_test.drop('fetal_health', axis=1)
y_train = df_train['fetal_health']
y_test = df_test['fetal_health']

# 5.
print("Training set shape:", X_train.shape)
print("Testing set shape:", X_test.shape)


# 7.
# Calculate percentages of class 0 in each dataset
# Initial dataset
initial_class_0_percent = (loadDF['fetal_health'] == 0).mean() * 100

# Training set
train_class_0_percent = (y_train == 0).mean() * 100

# Testing set
test_class_0_percent = (y_test == 0).mean() * 100

print(f"Percentage of class 0 in initial dataset: {initial_class_0_percent:.
print(f"Percentage of class 0 in training set: {train_class_0_percent:.2f}%"
print(f"Percentage of class 0 in testing set: {test_class_0_percent:.2f}%")
```

```
Training set shape: (967, 8)
Testing set shape: (323, 8)
Percentage of class 0 in initial dataset: 63.80%
Percentage of class 0 in training set: 63.81%
Percentage of class 0 in testing set: 63.78%
```

6. **Explain the purpose of this slightly complicated procedure. Why did we first split into the two classes? Why did we then split into a training and a testing set?** We split it into two classes to keep the same class distribution in both training and testing set. We basically do stratified sampling. If we did not do this we might sample 80% of class 0 and only 20% of class 1.

   We split into training and testing, to test our model how it generalizes to unseen data.

7. **What is the share (in percent) of samples with class 0 label in test and training set, and in the intial data set?** Percentage of class 0 in initial dataset: 63.80% Percentage of class 0 in training set: 63.81% Percentage of class 0 in testing set: 63.78%

Convert data to numpy arrays and shuffle the training data

Many machine learning models (including those you will work with later in the assignment) will not accept DataFrames as input. Instead, they will only work if you pass numpy arrays containing the data. Here, we convert the DataFrames `X_train`, `X_test`, `y_train`, and `y_test` to numpy arrays `X_train`, `X_test`, `y_train`, and `y_test`.

Moreover we shuffle the training data. This is important because the training data is currently ordered by class. In Part IV, we use the first n samples from the training set to train the classifiers. If we did not shuffle the data, the classifiers would only be trained on samples of class 0.

Nothing to be done here, just execute the cell.

In [15]:
```
# convert to numpy arrays
X_train = X_train.to_numpy()
X_test = X_test.to_numpy()
y_train = y_train.to_numpy()
y_test = y_test.to_numpy()

# shuffle training data
np.random.seed(42) # for reproducibility
shuffle_index = np.random.permutation(len(X_train)) # generate random indice
X_train, y_train = X_train[shuffle_index], y_train[shuffle_index] # shuffle
```

Part III: Scaling the data

1. Standardize the training *and* test data so that each feature has a mean of 0 and a standard deviation of 1.
2. Check that the scaling was successful
   - by printing the mean and standard deviation of each feature in the scaled training set
   - by putting the scaled training set into a DataFrame and make a violin plot of the data

Hint: use the `axis` argument to calculate mean and standard deviation column-wise.

Important: Avoid data leakage!

More hints:

1. For each column, subtract the mean (μ) of each column from each value in the column
2. Divide the result by the standard deviation (σ) of the column

(You saw how to do both operations in the lecture. If you don't remember, you can look

it up in Canvas files.)

Mathematically (in case this is useful for you), this transformation can be represented for each column as follows:

$$X_{\text{scaled}} = \frac{(X - \mu)}{\sigma}$$

where:

- (X_{scaled}) are the new, transformed column values (a column-vector)
- (X) is the original values
- (μ) is the mean of the column
- (σ) is the standard deviation of the column

In [16]:
```
# Insert your code below
# ======================

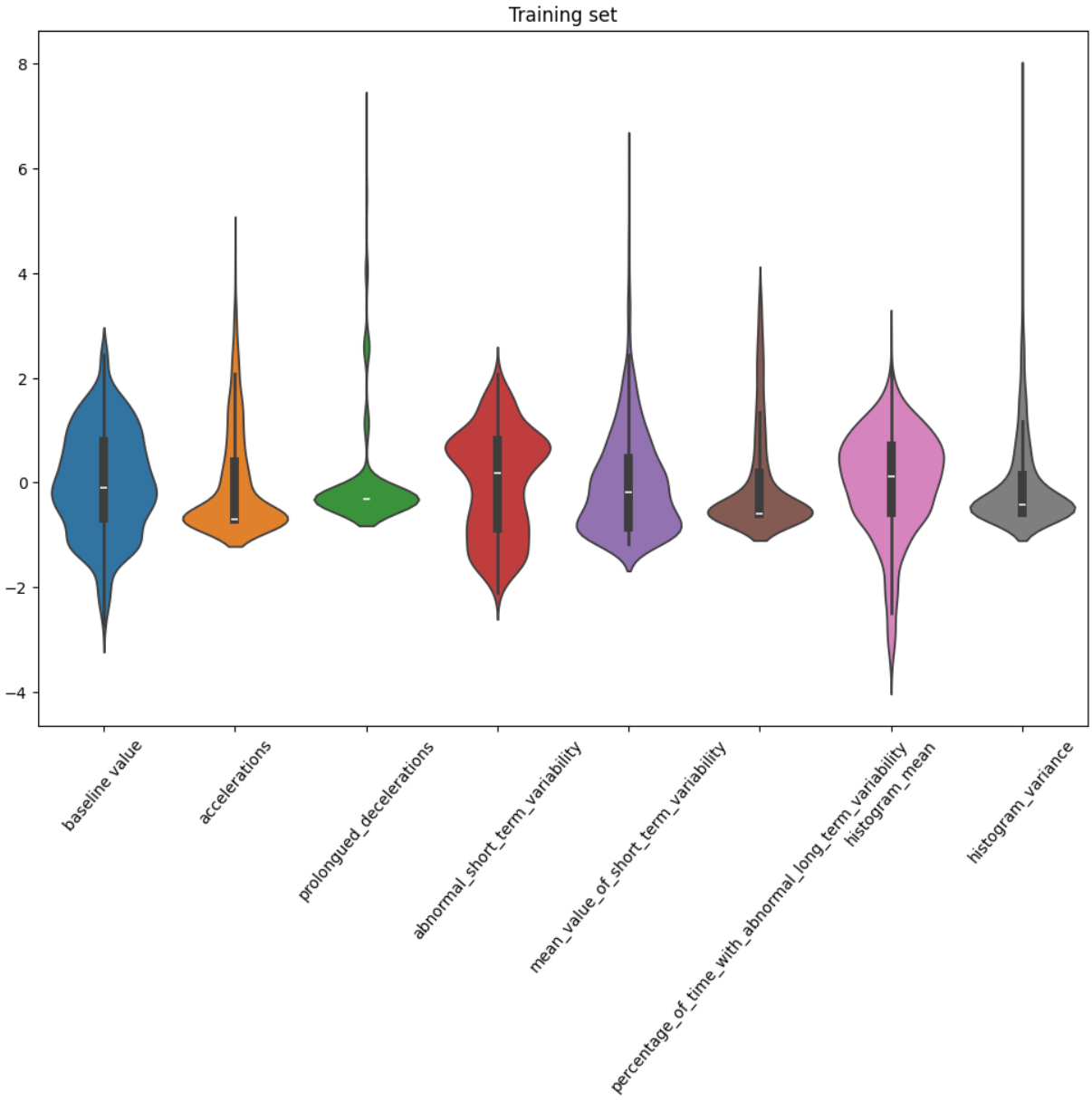
# 1. Standardize the data
X_train_scaled = (X_train - X_train.mean(axis=0)) / X_train.std(axis=0)
X_test_scaled = (X_test - X_test.mean(axis=0)) / X_test.std(axis=0)


# 2. Check if standardized and show violin plot
print("Mean of scaled training set:", X_train_scaled.mean(axis=0))
print("Standard deviation of scaled training set:", X_train_scaled.std(axis=


# From numpy arrays to pandas dataframes
df_X_train_scaled = pd.DataFrame(X_train_scaled, columns=df_train.columns[:-


# Violin plot:
plt.figure(figsize=(12, 8))
sns.violinplot(data=df_X_train_scaled)
plt.title('Training set')
plt.xticks(rotation=50)
plt.show()
```

```
Mean of scaled training set: [-1.31803106e-16  4.56925087e-15 -2.96097744e-16
1.33869705e-16
 -2.12543989e-17 -2.86453614e-16 -2.93342278e-16 -7.18717284e-17]
Standard deviation of scaled training set: [1. 1. 1. 1. 1. 1. 1. 1.]
```



Part IV: Training and evaluation with different dataset sizes and training times

Often, a larger dataset size will yield better model performance. (As we will learn later, this usually prevents overfitting and increases the generalization capability of the trained model.) However, collecting data is usually rather expensive.

In this part of the exercise, you will investigate

- how the model performance changes with varying dataset size
- how the model performance changes with varying numbers of epochs/iterations of the optimizer/solver (increasing training time).

For this task (Part IV), use the `Adaline`, `Perceptron`, and `LogisticRegression` classifier from the `mlxtend` library. All use the gradient descent (GD) algorithm for

training.

Important: Use a learning rate of `1e-4` (`0.0001`) for all classifiers, and use the argument `minibatches=1` when initializing `Adaline` and `LogisticRegression` classifier (this will make sure it uses GD). For all three classifiers, pass `random_seed=42` when initializing the classifier to ensure reproducibility of the results.

Model training

Train the model models using progressively larger subsets of your dataset, specifically: first 50 rows, first 100 rows, first 150 rows, ..., first 650 rows, first 700 rows (in total 14 different variants).

For each number of rows train the model with progressively larger number of epochs: 2, 7, 12, 17, ..., 87, 92, 97 (in total 20 different model variants).

The resulting $14 \times 20 = 280$ models obtained from the different combinations of subsets and number of epochs. An output of the training process could look like this:

```
Model (1) Train a model with first 50 rows of data for 2
epochs
Model (2) Train a model with first 50 rows of data for 7
epochs
Model (3) Train a model with first 50 rows of data for 12
epochs
...
Model (21) Train a model with first 100 rows of data for 2
epochs
Model (22) Train a model with first 100 rows of data for 7
epochs
...
Model (279) Train a model with first 700 rows of data for 92
epochs
Model (280) Train a model with first 700 rows of data for 97
epochs
```

Model evaluation

For each of the 280 models, calculate the **accuracy on the test set** (do **not** use the score method but compute accuracy yourself). Store the results in the provided 2D numpy array (it has 14 rows and 20 columns). The rows of the array correspond to the different dataset sizes, and the columns correspond to the different numbers of epochs.

Tasks

1. Train the 280 Adaline classifiers as mentioned above and calculate the accuracy for each of the 280 variants.
2. Generalize your code so that is doing the same procedure for all three classifiers: `Perceptron`, `Adaline`, and `LogisticRegression` after each other. Store the result for all classifiers. You can for example use an array of shape $3 \times 14 \times 20$ to store the accuracies of the three classifiers.

Note that executing the cells will take some time (but on most systems it should not be more than 5 minutes).

In [ ]:
```
# Insert your code below
# ======================
# Train and evaluate all model variants
from mlxtend.classifier import Adaline, Perceptron, LogisticRegression

classifiers = [Adaline, Perceptron, LogisticRegression]
#classifier_names = ['Adaline', 'Perceptron', 'LogisticRegression'] # Respec

# Create array to store accuracies (3 classifiers x 14 sample sizes x 20 epo
accuracies = np.zeros((3, 14, 20))

dataset_sizes = range(50, 701, 50)  # 14 values from 50 to 700
epoch_values = range(2, 98, 5)  # 20 values from 2 to 97


model_number = 0
# Loops 14 * 20 = 280 times, for every 14 sample sizes it loops through 20 e
# 280 Times for each classifier = 280 * 3 = 840 Total models
for i, nrows in enumerate(dataset_sizes):
    for j, epochs in enumerate(epoch_values):

        for clf_idx, clf_class in enumerate(classifiers):
            if clf_class == Perceptron:
                clf = clf_class(eta=0.0001, random_seed=42, epochs=epochs)
            else:
                clf = clf_class(eta=0.0001, random_seed=42, epochs=epochs, m

            # Train
            clf.fit(X_train_scaled[:nrows], y_train[:nrows])  # Select rows

            # Predict and calculate accuracy
            y_pred = clf.predict(X_test_scaled)
            accuracy = np.mean(y_pred == y_test)

            # Store accuracy
            accuracies[clf_idx, i, j] = accuracy
            model_number += 1
            #print(f"Model {str(classifiers[clf_idx].__name__)}({model_numbe
```

Performance visualization

Plot the performance measure for all classifiers (accuracy on the test set; use the result array from above) of all the 280 variants for each classifier in a total of three heatmaps using, for example `seaborn` or `matplotlib` directly.

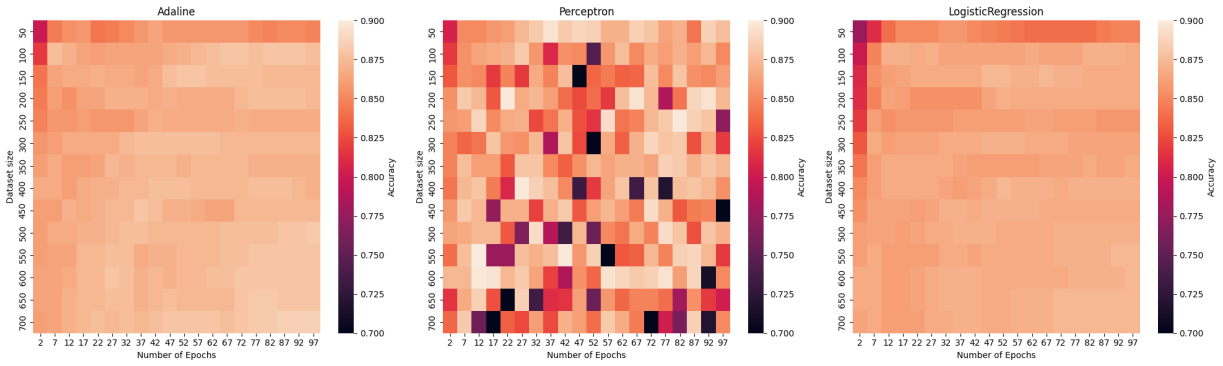
The color should represent the accuracy on the test set, and the x and y axes should represent the number of epochs and the dataset size, respectively. Which one is x and which one is y is up to you to decide. Look in the example output at the top of the assignment for inspiration for how the plot could look like and how it could be labeled nicely. (But use the correct numbers corresponding to your dataset sizes and number of epochs.)

In [27]:
```
# Insert your code below
# ======================
fig, axes = plt.subplots(1, 3, figsize=(20, 6))


for idx, ax in enumerate(axes):
    sns.heatmap(accuracies[idx],
                ax=ax,
                vmin=0.7,  # minimum accuracy to show
                vmax=0.90,  # maximum accuracy to show
                xticklabels=epoch_values,
                yticklabels=dataset_sizes,
                cbar_kws={'label': 'Accuracy'})

    ax.set_title(f'{str(classifiers[idx].__name__)}')  # Name property of th
    ax.set_xlabel('Number of Epochs')
    ax.set_ylabel('Dataset size')

plt.tight_layout()
plt.show()
```



Part V: Some more plotting

For the following cell to execute you need to have the variable `X_test_scaled` with all samples of the test set and the variable `y_test` with the corresponding labels. Complete at least up until Part III. Executing the cell will plot something.

1. Add code comments explaining what the lines are doing

2. What is the purpose of the plot?
3. Describe all components of the subplot and then comment in general on the entire plot. What does it show? What does it not show?

2. The plot visualizes the decision boundaries of a logistical regression across all pairs of features in the dataset. The 64 subplots show how two specific features change the models classification decision. Can show us what feature combinations that create clear seperation between the two classes. we can find out what features are more important.

3. The colored background, shows probability predicted by the model for class 1 across. Intensity indicates probability strength Black line, is the decision boundry, where the model predicts a 50% probability, (one side is class 0, other class 1) Blue triangles, are data points in class 0 Yellow cirles, data points in class 1

In [24]:
```
# Train and a logistic regression model with 300 epochs and learning rate 0.
clf = LogisticRegression(eta = 0.0001, epochs = 300, minibatches=1, random_s
clf.fit(X_test_scaled, y_test)

fig, axes = plt.subplots(8, 8, figsize=(30, 30))
# Loop through the two features index, for the  8x8 grid of plots
for i in range(0, 8):
    for j in range(0, 8):
        feature_1 = i
        feature_2 = j
        ax = axes[i, j]

        ax.set_xlabel(f"Feature {feature_1}")
        ax.set_ylabel(f"Feature {feature_2}")

    # Min and Max value of each feature
        mins = X_test_scaled.min(axis=0)
        maxs = X_test_scaled.max(axis=0)

    # Create 100 evenly spaced points between min and max for each feature
        x0 = np.linspace(mins[feature_1], maxs[feature_1], 100)
        x1 = np.linspace(mins[feature_2], maxs[feature_2], 100)

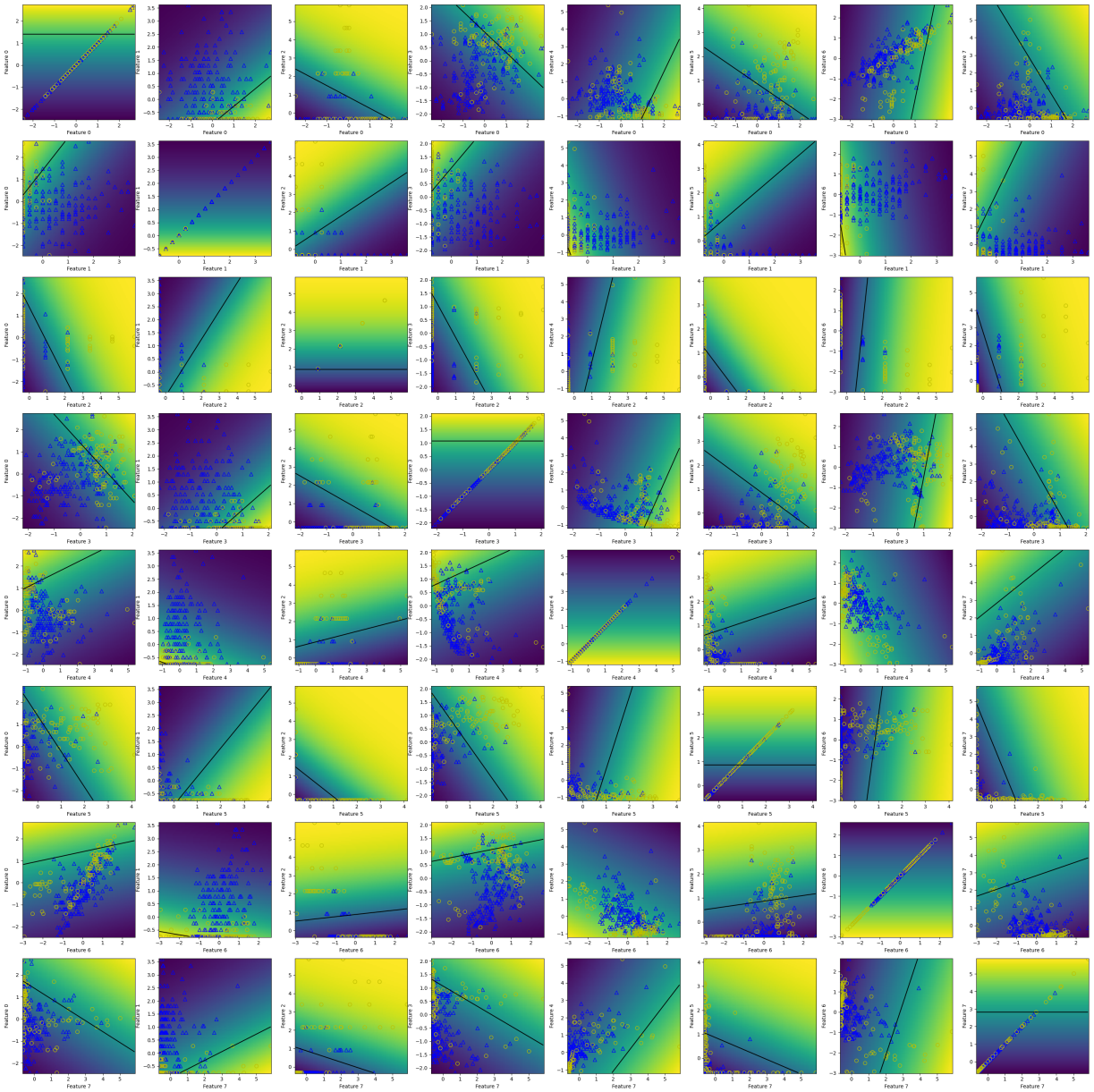

        X0, X1 = np.meshgrid(x0, x1)  # 2D grid
        X_two_features = np.c_[X0.ravel(), X1.ravel()]
        X_plot = np.zeros(shape=(X_two_features.shape[0], X_test_scaled.shap

    # Fill in values for current feature pair
        X_plot[:, feature_1] = X_two_features[:, 0]
        X_plot[:, feature_2] = X_two_features[:, 1]

    # Get probability predictions for all grid points
        y_pred = clf.predict_proba(X_plot)
        Z = y_pred.reshape(X0.shape)
```

```
            ax.pcolor(X0, X1, Z)  # Color map
            ax.contour(X0, X1, Z, levels=[0.5], colors='k')  # Draws the decisio
        # Set class 0 as blue triangles
            ax.scatter(X_test_scaled[y_test == 0, feature_1], X_test_scaled[y_te
        # Set class 1 as yellow circles
            ax.scatter(X_test_scaled[y_test == 1, feature_1], X_test_scaled[y_te

fig.tight_layout()
plt.show()
```



Part VI: Additional discussion

Part I:

1. What kind of plots did you use to visualize the raw data, and why did you choose these types of plots?

Part II:

1. What happens if we don't shuffle the training data before training the classifiers like in Part IV?
2. How could you do the same train/test split (Point 1.-4.) using scikit-learn?

Part IV:

1. How does increasing the dataset size affect the performance of the logistic regression model? Provide a summary of your findings.
2. Describe the relationship between the number of epochs and model accuracy
3. Which classifier is much slower to train and why do you think that is?
4. One classifier shows strong fluctuations in accuracy for different dataset sizes and number of epochs. Which one is it and why do you think this happens?

Assignment Answers

I.

1. A. Histogram with KDE why?
    - helps visualize the distribution, skewness, and spread of continous variables.
    - the KDE provide us with a smoothed out estimate of the distribution, making patterns more apparent.

   B. Bar plot why?
    - Effectively shows class imbalance and frequency of each category.
    - Useful for understanding how many samples belong to each class.

II.

1. Without shuffling the data, the model will be trained on data in an ordered way, and it might be biased towards a class and perform worse.

2. ```
   X_train, X_test, y_train, y_test = train_test_split(
       X, y,
       test_size=0.25,    # Test 25% Train 75% split
       random_state=42,
       stratify=y,
   )
   ```

IV.

1. The performance of the logistic regression model improves steadily with the dataset size, but it does not improve as much as the other models.
   ```

2.
   - Logistic regression seems to be very stable across epochs, reaches its ceiling and maintains performance.
   - Adaline has a gradual increase in performance across epochs.
   - Perceptron has a very high variance, no improvement with more epochs, performs well sometimes and poorly other times.
3. Adaline is likely the slowest classifier because:
   - It uses gradient descent which requires computing gradients for all samples
   - Makes smaller, more careful weight adjustments
4. The Perceptron has strong fluctuations, with both dataset size and epochs. Might be because it uses a simple binary decision rule that changes the classification a lot when weights change. The other models optimize continuous error functions.

In [ ]: